eCommerce Debug Bot – Step-by-Step Build Guide

Oct 27, 2026 · @Sunil

Before you start

Follow the phases in order; each one ends with a **Verify** step, so don't move on until it passes. The runner, tools, secrets and databases stay on your machine. Only what the AI tool asks for (knowledge files and masked, compressed traces) goes to your company-approved AI tool.

The guide assumes your eCommerce backend is a Node/TypeScript repo called `ecom-backend` using Hapi (an API server plus cron-driven service workers), axios and Postgres. Where your stack differs (NestJS, Prisma, Mongo, fetch), the step says what to swap.

Prerequisites

Item	Minimum	Check command
OS	macOS 13+, Ubuntu 22.04+, or Windows 11 with WSL2	—
RAM	8 GB (16 GB recommended)	—
Disk	10 GB free	`df -h`
Node.js	20 LTS or newer	`node -v`
Docker Desktop / Docker Engine	24+	`docker -v`
git	any recent	`git --version`
Approved AI tool	must support MCP servers and agent mode (see Phase 1)	—

If Node is missing, install it with nvm:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh |
bash
# restart the terminal, then:
nvm install 20
nvm use 20
node -v   # should print v20.x
```

Folder layout you will end up with

```
~/work/
  ecom-backend/            # your existing repo (small code changes in Phase
6)
    .knowledge/            # generated + hand-written knowledge (Phases 4–5)
    AGENTS.md              # pointer for any AI tool
  ecom-debug-bot/          # NEW tooling repo you create in Phase 3
    src/extract/           # code extractor
    src/mcp/               # MCP server (the bot's tools)
    src/profile/           # data profiler
    src/search/            # semantic search
    config/environments.yaml
    .env.stage             # secrets, never committed
```

Keeping the bot in a separate repo means your production code only gains the small explain-mode change; everything else lives outside it.

Phase 1 — Check your approved AI tool

Your company-approved AI tool is the bot's "brain". It reads the question, calls the gateway's tools (Phase 7) and explains the results. You don't install or run any model yourself.

What the tool must support

Capability	Why	How to check
MCP servers (local, stdio)	The gateway is an MCP server	Look for "MCP" in the tool's settings or docs
Agent mode	Lets it call several tools in a row	A mode named Agent, Composer or similar

Capability	Why	How to check
Rules / instructions file	Holds the bot's instructions (Phase 7.5)	e.g. `.github/copilot-instructions.md`, `.cursor/rules/`, `CLAUDE.md`
MCP allowed by your org	Some admins switch MCP off	Ask your admin, or try adding a server

Tools that meet all four today include GitHub Copilot in VS Code (agent mode), Cursor, Claude Code, Windsurf and JetBrains AI Assistant. Setup for the common ones is in Phase 7.4.

1.1 Confirm local MCP works (do this first, ~5 minutes)

Before building anything, prove that your AI tool can start an MCP server on your machine and call it. This tiny test server has one tool, `ping`, and touches no code or data.

1. **Create the test server:**

```
mkdir -p ~/work/mcp-hello && cd ~/work/mcp-hello
npm init -y && npm pkg set type=module
npm i @modelcontextprotocol/sdk zod
npm i -D tsx
```

Create `~/work/mcp-hello/hello.ts`:

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from
"@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";
import os from "node:os";

const server = new McpServer({ name: "hello-check", version: "0.0.1" });

server.registerTool("ping", {
  description: "Returns pong with this machine's hostname, to prove a
local MCP server works.",
  inputSchema: { name: z.string().default("world") },
}, async ({ name }) => ({
  content: [{ type: "text", text: `pong, ${name}, from ${os.hostname()} at
${new Date().toISOString()}` }],
}));

await server.connect(new StdioServerTransport());
console.error("hello-check MCP server running");   // stderr only
```

2. **Check it starts on its own:**

```
npx @modelcontextprotocol/inspector npx tsx ~/work/mcp-hello/hello.ts
```

   In the Inspector page, click **Connect → List Tools → ping → Run**. You should see `pong, world, from <your machine>`. Close the Inspector.

3. **Register it in your AI tool** using the same file format as Phase 7.4 for your tool, with server name `hello-check`, command `npx`, and args `["tsx", "/Users/you/work/mcp-hello/hello.ts"]` (use your absolute path). For example, in VS Code with Copilot, put this in the workspace's `.vscode/mcp.json`:

```
{
  "servers": {
    "hello-check": { "type": "stdio", "command": "npx", "args": ["tsx",
"/Users/you/work/mcp-hello/hello.ts"] }
  }
}
```

4. **Call it from the AI tool.** In agent mode, ask: "Use the ping tool from hello-check with name Sunil." Approve the call when asked.

Pass: the reply contains `pong, Sunil, from <your machine's hostname>`. That proves all four requirements in the table above: MCP is allowed, local stdio servers start, agent mode calls tools, and results come back. Continue with Phase 2.

If it fails, find your case here before going further:

What you see	What it means	Next step
No MCP option in the tool's settings	MCP is disabled by your org or not in your plan	Ask your admin; the rest of this guide needs it
Only servers from an approved list / registry can be added	Org allows hosted or listed servers only, not local ones	Stop: local MCP isn't allowed. Request a POC exception before continuing
Server shows an error or red status	Tool can't find `npx`, or the path is wrong	Use the full path from `which npx`; check the file path; restart the editor
Server is green but the tool never calls `ping`	Not in agent mode, or the tool is switched off	Switch to Agent mode; enable `hello-check` in the tools list
Works in the Inspector but not in the tool	Tool runs with a different Node or PATH	Set `command` to the absolute path of `npx`

When it passes, remove `hello-check` from the tool's config (or keep it as a quick health check) and note which config file your tool uses; you'll reuse it in Phase 7.4.

1.2 Find the instructions file

Find where your tool reads project instructions (see the table above). You'll put the bot's rules there in Phase 7.5.

Phase 2 — Agree what data may leave your machine

With a cloud AI tool, everything a tool call returns is sent to the vendor. Get this cleared once, before you connect any environment.

Data	Leaves your machine?	Control
Source code and `.knowledge/` files	Yes, when read	Usually already covered by the tool's approval
Traces, API and DB results from stage/qa	Yes, as tool results	Masked + truncated by the gateway (Phase 6.5, 7.1)
Secrets (tokens, DB URLs)	**No**	Stay in `.env.*`; never returned by any tool
Production data	**No**	Not in `ALLOWED_ENVS` (Phase 6.5)

1. **Ask your security team one question:** "May masked stage/qa responses (SKUs, prices, inventory, offer configs, no customer data) be sent to [approved tool] as tool results?" Keep the answer in writing.
2. **Extend the PII mask** in Phase 6.5 with any other field names your stage data uses for customer details (for example `customerId`, `loyaltyNumber`, `dob`).
3. **Keep production out.** Leave `prod` out of `ALLOWED_ENVS` unless security explicitly approves it.
4. **Disable training on your data** if your tool's org settings offer it (enterprise plans usually do this by default).

Phase 3 — Create the tooling project

This repo holds the extractor, the bot's MCP server, the profiler and search.

1. **Create and initialise it:**

```
mkdir -p ~/work/ecom-debug-bot && cd ~/work/ecom-debug-bot
git init
npm init -y
npm pkg set type=module
```

2. **Install dependencies:**

```
npm i ts-morph yaml zod dotenv axios fast-glob @modelcontextprotocol/sdk
npm i -D typescript tsx @types/node
```

Package	Why
ts-morph	Reads your TypeScript code as a syntax tree
yaml	Writes compact knowledge files
zod	Validates tool inputs
dotenv	Loads per-environment secrets
axios	Calls downstream APIs
fast-glob	Finds files
@modelcontextprotocol/sdk	Exposes tools to your AI tool
tsx	Runs `.ts` files directly

3. **Create `tsconfig.json`:**

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

4. **Create the folders and ignore secrets:**

```
mkdir -p src/extract src/mcp src/profile src/search config
printf "node_modules\ndist\n.env*\n.lancedb\n" > .gitignore
```

5. **Point the tools at your backend** with a `.env` file in the tooling repo:

```
ECOM_REPO=/Users/you/work/ecom-backend
```

Verify: `npx tsx -e "console.log('ok')"` prints `ok`.

Phase 4 — Generate knowledge from your code

A script reads your backend's TypeScript (no LLM involved) and writes compact facts into `ecom-backend/.knowledge/generated/`. Then your approved AI tool writes a short summary per module from those facts.

4.1 Create the extractor

Create `src/extract/index.ts`:

```
import "dotenv/config";
import { Project, SyntaxKind, Node, SourceFile } from "ts-morph";
import { mkdirSync, writeFileSync } from "node:fs";
import { createHash } from "node:crypto";
import path from "node:path";
import YAML from "yaml";

const REPO = process.env.ECOM_REPO!;
const OUT = path.join(REPO, ".knowledge", "generated");
mkdirSync(OUT, { recursive: true });

const project = new Project({ tsConfigFilePath: path.join(REPO,
"tsconfig.json") });
const files = project.getSourceFiles().filter(
  (f) => !/node_modules|\.(spec|test)\.ts$|\.d\.ts$/.test(f.getFilePath())
);

const rel = (p: string) => path.relative(REPO, p);
const HTTP = new Set(["get", "post", "put", "patch", "delete"]);
const NEST = new Set(["Get", "Post", "Put", "Patch", "Delete"]);

const routes: any[] = [];
const outbound: any[] = [];
const envVars: Record<string, Set<string>> = {};
const symbols: Record<string, any[]> = {};
const callGraph: Record<string, string[]> = {};
const manifest: Record<string, string> = {};

// Name of the function/method that contains a node
function owner(node: Node): string {
  for (const a of node.getAncestors()) {
    if (Node.isFunctionDeclaration(a) || Node.isMethodDeclaration(a)) {
      const cls =
```

```
a.getFirstAncestorByKind(SyntaxKind.ClassDeclaration)?.getName();
      return (cls ? cls + "." : "") + (a.getName() ?? "anonymous");
    }
    if (Node.isVariableDeclaration(a)) return a.getName();
  }
  return "<module>";
}

function scanFile(sf: SourceFile) {
  const file = rel(sf.getFilePath());
  manifest[file] =
createHash("sha1").update(sf.getFullText()).digest("hex").slice(0, 12);

  // 1) Express routes + outbound HTTP calls
  for (const call of sf.getDescendantsOfKind(SyntaxKind.CallExpression)) {
    const expr = call.getExpression();
    if (!Node.isPropertyAccessExpression(expr)) continue;
    const method = expr.getName();
    const target = expr.getExpression().getText();
    const [first] = call.getArguments();
    if (!HTTP.has(method)) continue;

    if (/^(app|router|\w+Router)$/.test(target) && first &&
Node.isStringLiteral(first)) {
      routes.push({ method: method.toUpperCase(), path:
first.getLiteralValue(), file,
        handler: call.getArguments().at(-1)?.getText().slice(0, 80) });
    } else if (/axios|http|client|api/i.test(target)) {
      outbound.push({ caller: owner(call), via: target, method:
method.toUpperCase(),
        url: first?.getText().slice(0, 150), file, line:
call.getStartLineNumber() });
    }
  }

  // 2) NestJS routes (@Get('path') on methods)
  for (const m of sf.getDescendantsOfKind(SyntaxKind.MethodDeclaration)) {
    for (const d of m.getDecorators()) {
      if (NEST.has(d.getName())) {
        routes.push({ method: d.getName().toUpperCase(), path:
d.getArguments()[0]?.getText() ?? "''",
          file, handler: owner(m.getBody() ?? m) });
      }
```

```
    }
  }

  // 3) Environment variables (process.env.X)
  for (const pa of
sf.getDescendantsOfKind(SyntaxKind.PropertyAccessExpression)) {
    if (pa.getExpression().getText() === "process.env") {
      (envVars[pa.getName()] ??= new Set()).add(file);
    }
  }

  // 4) Exported symbols with signatures + JSDoc
  for (const [name, decls] of sf.getExportedDeclarations()) {
    for (const d of decls) {
      const entry: any = { name, kind: d.getKindName() };
      if (Node.isFunctionDeclaration(d)) {
        entry.sig = `(${d.getParameters().map((p) => p.getText()).join(",
")}): ${d.getReturnType().getText(d)}`;
        entry.doc = d.getJsDocs().map((j) =>
j.getDescription().trim()).join(" ") || undefined;
      }
      if (Node.isClassDeclaration(d)) entry.methods = d.getMethods().map((m)
=> m.getName());
      if (Node.isInterfaceDeclaration(d) || Node.isTypeAliasDeclaration(d))
entry.def = d.getText().slice(0, 400);
      (symbols[file] ??= []).push(entry);
    }
  }

  // 5) Call graph: function -> names it calls
  const fns = [...sf.getFunctions(), ...sf.getClasses().flatMap((c) =>
c.getMethods())];
  for (const fn of fns) {
    const key = `${file}#${owner(fn.getBody() ?? fn)}`;
    const callees = new Set(
      fn.getDescendantsOfKind(SyntaxKind.CallExpression).map((c) =>
c.getExpression().getText().split("(")[0].slice(0, 60))
    );
    callGraph[key] = [...callees];
  }
}

files.forEach(scanFile);
```

```
const env = Object.fromEntries(Object.entries(envVars).map(([k, v]) => [k,
[...v]]));
writeFileSync(path.join(OUT, "routes.yaml"), YAML.stringify(routes));
writeFileSync(path.join(OUT, "downstream-calls.yaml"),
YAML.stringify(outbound));
writeFileSync(path.join(OUT, "env-vars.yaml"), YAML.stringify(env));
writeFileSync(path.join(OUT, "symbols.yaml"), YAML.stringify(symbols));
writeFileSync(path.join(OUT, "call-graph.json"), JSON.stringify(callGraph));
writeFileSync(path.join(OUT, "manifest.json"), JSON.stringify(manifest, null,
1));
console.log(`files=${files.length} routes=${routes.length}
outbound=${outbound.length} env=${Object.keys(env).length}`);
```

Adjust for your stack: if your HTTP client is a wrapper (for example `this.inventoryClient.get(...)`), the regex `/axios|http|client|api/i` already catches names containing "client" or "api"; add your wrapper's name if it differs. For `fetch(url)`, add a check for `expr.getText() === "fetch"`.

Hapi routes: Hapi declares routes as `server.route({ method, path, handler })` objects, which this extractor doesn't parse. You don't need it to: Phase 7.2 writes the real route table from `server.table()` into `generated/hapi-routes.yaml`. Keep this extractor for outbound calls, env vars, symbols and the call graph.

4.2 Run it

Add scripts to the tooling repo's `package.json`:

```
npm pkg set scripts.extract="tsx src/extract/index.ts"
npm run extract
```

Verify: the console prints non-zero counts, and `ecom-backend/.knowledge/generated/` contains six files. Open `downstream-calls.yaml` and check that your inventory, catalog and offer API calls appear. If a known call is missing, adjust the regex as above.

4.3 Add the module dependency graph

Run inside the backend repo:

```
cd ~/work/ecom-backend
npx madge --ts-config tsconfig.json --extensions ts --json src >
.knowledge/generated/deps.json
npx madge --ts-config tsconfig.json --extensions ts --circular src   # bonus:
lists circular imports
```

4.4 Write module summaries with your AI tool

Open `ecom-backend` in your AI tool, switch to agent mode, and run this prompt once per module folder under `src/` (for example `offers`, `inventory`, `pricing`, `workers`). Save it as `.knowledge/prompts/module-summary.md` so the whole team uses the same one.

```
Write .knowledge/modules/{MODULE}.md for the module in src/{MODULE}/.

Inputs (read these first, in this order):
1. .knowledge/generated/symbols.yaml - entries for files under src/{MODULE}/
2. .knowledge/generated/downstream-calls.yaml - entries for files under
src/{MODULE}/
3. The source files themselves, only where the YAML isn't enough

Use exactly these sections:
# {MODULE}
## Purpose (2 sentences)
## Entry points (bullets: file#function -> one line)
## Business rules (bullets, each ending with its source file#function)
## Downstream dependencies (bullets: API or table + why it's called)
## Failure modes (bullets: what goes wrong and the reason code, if any)

Rules: only facts from the code; max 350 words; no code blocks; mark anything
unclear as "UNCLEAR:".
At the very top, add: <!-- source_hash: {the value listed for this module in
.knowledge/STALE.md} -->
```

Replace `{MODULE}` before sending. Before the first run, create the stale-summary script from Phase 10.1 and run npm run stale in the tooling repo: it lists every module with the hash its summary should carry. Most tools can do several modules in one run if you list them, but one at a time gives better summaries.

Verify: open `.knowledge/modules/offers.md`. Check every business rule against the code, fix anything wrong, and resolve every `UNCLEAR:` line. These files are reviewed by you, not trusted blindly, and committed with the code.

Phase 5 — Write the hand-curated knowledge

Generated files tell an AI *what exists*; these files tell it *how the business works*. Spend real time here, because they are what make answers correct. All go in `ecom-backend/.knowledge/`.

5.1 `INDEX.md` — the only file always loaded (keep under ~60 lines)

```
# Knowledge index (read this first, then load only what you need)

## System in one paragraph
Node/TS backend serving catalog, pricing, offers, inventory and cart for web
+ app.

## Where to look
| Question about | Read |
|---|---|
| Why an offer is/isn't applied | rules/offer-eligibility.yaml,
modules/offers.md |
| Inventory / availability | modules/inventory.md |
| Which API / URL per env | envs/environments.yaml, generated/downstream-
calls.yaml |
| DB tables and fields | data/data-dictionary.yaml |
| What a data job produces | data/job-outputs.yaml |
| Terms (ecomFlag, stackable...) | glossary.yaml |
| HTTP endpoints | generated/routes.yaml |
| Who calls what | generated/call-graph.json (large: search, don't load
whole) |

## Debugging tools available
reproduce_request, run_worker, list_routes, list_workers, call_api, query_db,
read_knowledge, search_code, find_job_traces
```

5.2 `AGENTS.md` at the repo root

Many AI coding tools read this file automatically.

```
# AGENTS.md
Before answering questions about this codebase, read .knowledge/INDEX.md and
load only the files it points to.
Prefer YAML knowledge files over reading source. Open source files only at
the file#function a rule cites.
Never modify .knowledge/generated/* by hand; run `npm run extract` in ecom-
debug-bot.
```

5.3 `rules/offer-eligibility.yaml` — the most important file

Write one entry per rule your offer engine applies, in the order it applies them. Open your offer code and copy each check's real condition.

```
version: 1
entry_point: src/offers/eligibility.ts#evaluateOffer
evaluation_order: [offer_active, ecom_flag, inventory, channel, stackability,
customer_segment]
rules:
  - id: offer_active
    check: offer.status == ACTIVE and now between offer.startDate and
offer.endDate (UTC)
    source: src/offers/rules/offerActive.ts#isActive
    data_from: offers-api GET /offers/{offerId}
    fail_reason: OFFER_INACTIVE
    common_causes: [end date passed in stage, timezone mismatch, offer not
published to stage]
  - id: ecom_flag
    check: product.ecomFlag == 'Y'
    source: src/offers/rules/ecomFlag.ts#isEcomEnabled
    data_from: catalog-api GET /products/{sku}
    fail_reason: OFFER_ECOM_FLAG_OFF
  - id: inventory
    check: availableQty >= config.offers.minQty (default 1) at the requested
store
    source: src/inventory/availability.ts#getAvailable
    data_from: inventory-api GET /inventory/{sku}?storeId={storeId}
    fail_reason: OFFER_NO_INVENTORY
  - id: stackability
    check: no other active offer on the SKU with stackable=false and higher
```

```
priority
    source: src/offers/stacking.ts#resolveStack
    data_from: offers-api GET /offers?sku={sku}&status=ACTIVE
    fail_reason: OFFER_NOT_STACKABLE
```

5.4 `envs/environments.yaml` — URLs only, never secrets

```
stage:
  catalog_api: https://catalog.stage.internal.example
  inventory_api: https://inventory.stage.internal.example
  offers_api: https://offers.stage.internal.example
  auth: { type: bearer, token_env: STAGE_API_TOKEN }
  db: { url_env: STAGE_DB_URL_READONLY }
qa:
  catalog_api: https://catalog.qa.internal.example
  inventory_api: https://inventory.qa.internal.example
  offers_api: https://offers.qa.internal.example
  auth: { type: bearer, token_env: QA_API_TOKEN }
  db: { url_env: QA_DB_URL_READONLY }
```

Copy the same file to `ecom-debug-bot/config/environments.yaml` (Phase 6.3 extends it for the runner). The `*_env` fields name variables held in `.env.stage` / `.env.qa`, so tokens never appear in knowledge or prompts.

5.5 `glossary.yaml`

```
ecomFlag: Catalog attribute; 'Y' means the product may be sold online. Set by
merchandising feed.
stackable: Offer attribute; false means it cannot combine with another offer
on the same SKU.
channel: WEB | APP | STORE; offers can target a subset.
```

Verify: paste `INDEX.md` into your AI tool's chat and ask "Which files would you read to explain why an offer is missing?" It should name the rules file and the offers module.

Phase 6 — Integrate once: the debug runner (Hapi + cron workers)

The runner boots your **real** Hapi server or any cron worker in a child process, configured for one environment (stage, qa…). It records every business function call (arguments + return), every downstream API response and every DB query, blocks all writes, and

returns a compact trace. No per-feature code: a new domain is one folder name in a regex.

For the offer question, the bot calls the runner with `GET /products/4455/offers?storeId=12` and reads back a trace like this:

```
GET /products/4455/offers?storeId=12 → 200 []
├ offers/eligibility#getCandidateOffers(["4455"]) → ["OFF-123","OFF-099"]
| ├ HTTP GET https://offers.stage…/offers?sku=4455 → 200 [{id:OFF-
123,stackable:false…}]
├ catalog/product#getProduct(["4455"]) → {ecomFlag:"Y",…}
├ inventory/availability#isAvailable(["4455","12"]) → false
| ├ HTTP GET https://inventory.stage…/inventory/4455?storeId=12 → 200
{availableQty:0}
├ offers/stacking#resolveStack([["OFF-123","OFF-099"]]) → ["OFF-099"]
└ offers/filter#applyEligibility(…) → []
```

The backend needs two small, one-time refactors (6.1, 6.2). Everything else lives in the tooling repo.

6.1 Backend: build the server without starting it

Hapi's `server.inject()` runs a request through all your plugins, auth, validation and handlers without opening a port. To use it, the server must be creatable without `start()`. Many Hapi projects already have this split for `@hapi/lab` tests; if yours does, reuse it.

`src/server.ts`:

```
import Hapi from "@hapi/hapi";
import { registerPlugins } from "./plugins";
import { registerRoutes } from "./routes";

export async function createServer() {
  const server = Hapi.server({ port: Number(process.env.PORT ?? 3000), host:
"0.0.0.0" });
  await registerPlugins(server);
  registerRoutes(server);
  await server.initialize();          // runs onPreStart etc.; ready for
inject(), no port opened
  return server;
}
```

`src/index.ts` (production entry, behaviour unchanged):

```
import { createServer } from "./server";
createServer().then((s) => s.start()).then(() => console.log("server
started"));
```

If a plugin starts background work on initialize (queue consumers, schedulers, cache warmers), skip it in debug mode:

```
if (process.env.DEBUG_RUNNER !== "true") startQueueConsumer();
```

6.2 Backend: a worker registry (separate "what" from "when")

Cron decides *when* a worker runs; the registry exposes *what* it does, so the runner can call any worker on demand, for one SKU, without waiting for the schedule.

`src/workers/registry.ts`:

```
import { syncInventory } from "./inventorySync";
import { publishOffers } from "./offerPublisher";
import { buildProductFeed } from "./productFeed";

export type WorkerArgs = { skus?: string[]; limit?: number; dryRun?: boolean
};

export const WORKERS: Record<string, (args: WorkerArgs) => Promise<unknown>>
= {
  "inventory-sync": syncInventory,
  "offer-publisher": publishOffers,
  "product-feed": buildProductFeed,
};
```

Your cron file keeps the schedule and calls the registry (shown with node-cron; agenda, bull or node-schedule work the same way):

```
import cron from "node-cron";
import { WORKERS } from "./registry";
cron.schedule("*/15 * * * *", () => WORKERS["inventory-sync"]({}));
cron.schedule("0 * * * *", () => WORKERS["offer-publisher"]({}));
```

Give each worker an optional `skus` / `limit` filter, so a debug run processes one product instead of the whole catalog. This is the only change inside worker code.

Build the backend with tsc. The runner loads the compiled `dist/` output (`npm run build`), because tsc-compiled CommonJS exports can be wrapped at load time. Check that `tsconfig.json` has `"module": "commonjs"` and `"sourceMap": true`.

6.3 Describe each environment for the runner

Replace `ecom-debug-bot/config/environments.yaml` with:

```
stage:
  backend_env_file: .env.backend.stage     # env vars your Hapi app reads at
boot
  services:                                # used by the call_api tool
    catalog: https://catalog.stage.internal.example
    inventory: https://inventory.stage.internal.example
    offers: https://offers.stage.internal.example
  auth: { token_env: STAGE_API_TOKEN }
  db_readonly_env: STAGE_DB_URL_READONLY
  allow_post_paths: ["/offers/search"]     # downstream POSTs that only read
qa:
  backend_env_file: .env.backend.qa
  services: { catalog: https://catalog.qa.internal.example }
  auth: { token_env: QA_API_TOKEN }
  db_readonly_env: QA_DB_URL_READONLY
```

`.env.backend.stage` = exactly the variables your deployment sets for stage (`NODE_ENV`, downstream URLs, DB URL, Redis…), with two changes: **the DB user is read-only** and **API tokens are read-only**. If you use the `config` npm package, `NODE_ENV=stage` plus `NODE_CONFIG_DIR=/Users/you/work/ecom-backend/config` is usually enough.

`.env.stage` holds the tooling's own secrets (`STAGE_API_TOKEN`, `STAGE_DB_URL_READONLY`). Both files are git-ignored by the `.env*` rule.

6.4 Install the runner's dependencies (tooling repo)

```
cd ~/work/ecom-debug-bot
npm i require-in-the-middle @opentelemetry/api @opentelemetry/sdk-node \
      @opentelemetry/sdk-trace-base @opentelemetry/auto-instrumentations-node
pg
npm i -D @types/pg
mkdir -p src/runner
```

6.5 Shared config + masking

`src/runner/config.ts`:

```
import { readFileSync } from "node:fs";
import path from "node:path";
import YAML from "yaml";
import dotenv from "dotenv";

export const ROOT = path.resolve(import.meta.dirname, "../..");
export const ALLOWED_ENVS = ["dev", "stage", "qa"] as const;      // no prod
until you trust it
export type Env = (typeof ALLOWED_ENVS)[number];

const all = YAML.parse(readFileSync(path.join(ROOT,
"config/environments.yaml"), "utf8"));
export function cfg(env: Env) {
  if (!ALLOWED_ENVS.includes(env) || !all[env]) throw new Error(`Env ${env}
not allowed or not configured`);
  return all[env];
}
export const secrets = (env: Env) =>
dotenv.parse(readFileSync(path.join(ROOT, `.env.${env}`)));
```

`src/runner/mask.ts`:

```
const PII =
/email|phone|firstName|lastName|address|customerName|card|password|token/i;
export function mask(v: unknown): unknown {
  if (Array.isArray(v)) return v.map(mask);
  if (v && typeof v === "object")
    return Object.fromEntries(Object.entries(v).map(([k, x]) => [k,
PII.test(k) ? "***" : mask(x)]));
  return v;
}
export const short = (v: unknown, n = 400) => {
  try { return JSON.stringify(mask(v))?.slice(0, n) ?? "undefined"; } catch {
return "[unserializable]"; }
};
```

6.6 The instrumentation (the "integrate once" core)

`src/runner/instrument.ts` must load before any backend code. It does four things: starts tracing into memory, wraps business functions, captures and guards axios calls, and guards Postgres writes.

```
import { NodeSDK } from "@opentelemetry/sdk-node";
import { InMemorySpanExporter, SimpleSpanProcessor } from
"@opentelemetry/sdk-trace-base";
import { getNodeAutoInstrumentations } from "@opentelemetry/auto-
instrumentations-node";
import { trace } from "@opentelemetry/api";
import { Hook } from "require-in-the-middle";
import { short } from "./mask.js";

// 1) Tracing into memory (DB, Redis, Hapi spans come from auto-
instrumentation)
export const exporter = new InMemorySpanExporter();
const sdk = new NodeSDK({
  serviceName: "debug-runner",
  spanProcessors: [new SimpleSpanProcessor(exporter)],
  instrumentations: [getNodeAutoInstrumentations({
    "@opentelemetry/instrumentation-fs": { enabled: false },
    "@opentelemetry/instrumentation-http": { enabled: false },   // axios is
captured below, with bodies
  })],
});
sdk.start();
export const shutdown = () => sdk.shutdown();
export const tracer = trace.getTracer("biz");
export const blocked: string[] = [];

// 2) Wrap every exported function in the watched folders of dist/
const BUILD = process.env.BUILD_DIR ?? "dist";
const WATCH = new RegExp(`/${BUILD}/(${process.env.WATCH ??
"offers|pricing|inventory|catalog|cart|promotions|workers|services"})/`);

new Hook(null, { internals: true }, (exports: any, name: string) => {
  const file = name.replace(/\\/g, "/");
  if (!WATCH.test(file) || !exports || typeof exports !== "object") return
exports;
  for (const key of Object.keys(exports)) {
    const fn = exports[key];
```

```
    if (typeof fn !== "function" || /^[A-Z]/.test(key)) continue;          //
skip classes
    const label = `${file.split(`/${BUILD}/`)[1].replace(/\.js$/,
"")}#${key}`;
    const wrapped = function (this: unknown, ...args: unknown[]) {
      return tracer.startActiveSpan(label, (span) => {
        span.setAttribute("biz", true);
        span.setAttribute("args", short(args));
        const done = (r: unknown) => { span.setAttribute("return", short(r));
span.end(); return r; };
        const failed = (e: any) => { span.setAttribute("error",
String(e?.message ?? e)); span.end(); throw e; };
        try {
          const r: any = fn.apply(this, args);
          return r && typeof r.then === "function" ? r.then(done, failed) :
done(r);
        } catch (e) { return failed(e); }
      });
    };
    try { exports[key] = wrapped; } catch { /* non-writable export: skip */ }
  }
  return exports;
});

// 3) axios: record every downstream response; block writes but keep the flow
running
new Hook(["axios"], (axios: any) => {
  const allowPost: string[] = JSON.parse(process.env.ALLOW_POST_PATHS ?? "
[]");
  const attach = (inst: any) => {
    inst.interceptors.request.use((c: any) => {
      const m = String(c.method ?? "get").toUpperCase();
      const url = `${c.baseURL ?? ""}${c.url ?? ""}`;
      if (!["GET", "HEAD"].includes(m) && !allowPost.some((p) =>
url.includes(p))) {
        blocked.push(`HTTP ${m} ${url} ${short(c.data, 300)}`);
        c.adapter = async () => ({ data: { debugRunner: "write blocked" },
status: 200, statusText: "BLOCKED", headers: {}, config: c });
      }
      return c;
    });
    const record = (c: any, status: unknown, body: unknown) =>
      trace.getActiveSpan()?.addEvent("downstream", {
```

```
        method: String(c?.method ?? "get").toUpperCase(), url: `${c?.baseURL
?? ""}${c?.url ?? ""}`,
        status: String(status), body: short(body, 600) });
    inst.interceptors.response.use(
      (r: any) => { record(r.config, r.status, r.data); return r; },
      (e: any) => { record(e.config, e.response?.status ?? "ERR",
e.response?.data ?? e.message); throw e; });
  };
  attach(axios);
  const create = axios.create;
  axios.create = function (...a: any[]) { const inst = create.apply(this, a);
attach(inst); return inst; };
  return axios;
});

// 4) Postgres: let reads through, record and skip writes
new Hook(["pg"], (pg: any) => {
  for (const C of [pg.Client, pg.Pool]) {
    const query = C.prototype.query;
    C.prototype.query = function (text: any, ...rest: any[]) {
      const sql = typeof text === "string" ? text : text?.text ?? "";
      if (!/^\s*(select|show|explain)\b/i.test(sql) && !/^\s*with\b(?!
[\s\S]*\b(insert|update|delete)\b)/i.test(sql)) {
        blocked.push(`SQL ${sql.replace(/\s+/g, " ").slice(0, 300)}`);
        const fake = { rows: [], rowCount: 0, command: "BLOCKED" };
        const cb = rest.find((r) => typeof r === "function");
        if (cb) return void cb(null, fake);
        return Promise.resolve(fake);
      }
      return query.call(this, text, ...rest);
    };
  }
  return pg;
});
```

Notes:

- Calls **between functions in the same file** don't pass through `exports`, so they show as one step. If a critical decision is buried inside a large function, extract it into its own exported function (see 6.9).

- Using `fetch` instead of axios? Re-enable `instrumentation-http` and `instrumentation-undici` (URLs and status, no bodies). Using MongoDB? Add a hook on `mongodb` that wraps `Collection.prototype` write methods (`insertOne`, `updateOne`, `deleteMany`…) the same way as `pg`.
- These guards keep the flow running. The read-only DB user and read-only tokens from 6.3 are the real safety net; keep both.

6.7 The child process: runs one job

`src/runner/child.ts`:

```
import { exporter, shutdown, blocked, tracer } from "./instrument.js";   //
MUST be the first import
import { createRequire } from "node:module";
import { writeFileSync } from "node:fs";
import path from "node:path";
import { compress } from "./compress.js";
import { short } from "./mask.js";

const job = JSON.parse(process.argv[2]);
const req = createRequire(import.meta.url);
const load = (p: string) => req(path.join(process.cwd(),
process.env.BUILD_DIR ?? "dist", p));

async function run(): Promise<unknown> {
  switch (job.kind) {
    case "request": {
      const server = await load("server.js").createServer();
      const res = await server.inject({ method: job.method ?? "GET", url:
job.url, headers: job.headers, payload: job.payload });
      await server.stop();
      return { status: res.statusCode, body: res.result ?? res.payload };
    }
    case "worker": {
      const { WORKERS } = load("workers/registry.js");
      if (!WORKERS[job.name]) throw new Error(`Unknown worker ${job.name}.
Known: ${Object.keys(WORKERS).join(", ")}`);
      return WORKERS[job.name]({ ...job.args, dryRun: true });
    }
    case "routes": {
      const server = await load("server.js").createServer();
      const routes = server.table().map((r: any) => ({ method:
```

```
r.method.toUpperCase(), path: r.path,
        description: r.settings.description, tags: r.settings.tags }));
      await server.stop();
      return routes;
    }
    case "workers":
      return Object.keys(load("workers/registry.js").WORKERS);
  }
  throw new Error(`Unknown job kind ${job.kind}`);
}

let result: unknown, error: string | undefined;
try {
  result = await tracer.startActiveSpan("job", async (span) => { try { return
await run(); } finally { span.end(); } });
} catch (e: any) { error = String(e?.stack ?? e).slice(0, 2000); }
await new Promise((r) => setTimeout(r, 300));        // let late async spans
finish

const listing = job.kind === "routes" || job.kind === "workers";
writeFileSync(job.out, JSON.stringify({
  result: listing ? result : short(result, 3000),
  error,
  trace: listing ? undefined : compress(exporter.getFinishedSpans()),
  blocked_writes: blocked,
}));
await shutdown();
process.exit(0);
```

The result goes to a file, not stdout, because your Hapi app's own logs also print to stdout.

6.8 Trace compression and the parent launcher

`src/runner/compress.ts` turns hundreds of spans into a short tree (typically 1–3k tokens):

```
import type { ReadableSpan } from "@opentelemetry/sdk-trace-base";

const sid = (s: ReadableSpan) => s.spanContext().spanId;
const pid = (s: any): string | undefined => s.parentSpanContext?.spanId ??
s.parentSpanId;
const t = (s: ReadableSpan) => s.startTime[0] * 1e9 + s.startTime[1];
```

```
function label(s: any): string | null {
  const a = s.attributes;
  if (a.biz) return `${s.name}(${a.args}) → ${a.error ? "THROW " + a.error :
a.return}`;
  const stmt = a["db.query.text"] ?? a["db.statement"];
  if (stmt) return `DB ${String(stmt).replace(/\s+/g, " ").slice(0, 200)}`;
  return null;                                    // Hapi internals, Redis
pings, etc. are folded away
}

export function compress(spans: ReadableSpan[], maxLines = 150): string {
  const ids = new Set(spans.map(sid));
  const kids = new Map<string, ReadableSpan[]>();
  for (const s of spans) { const p = pid(s); if (p && ids.has(p)) kids.set(p,
[...(kids.get(p) ?? []), s]); }
  const lines: string[] = [];
  const pad = (d: number) => "│ ".repeat(d) + "├ ";

  const walk = (s: any, depth: number) => {
    const l = label(s);
    const d = l ? depth + 1 : depth;
    if (l) lines.push(pad(depth) + l);
    for (const e of s.events ?? []) {
      if (e.name !== "downstream") continue;
      const x = e.attributes;
      lines.push(pad(d) + `HTTP ${x.method} ${x.url} → ${x.status}
${x.body}`);
    }
    const children = (kids.get(sid(s)) ?? []).sort((a, b) => t(a) - t(b));
    for (let i = 0; i < children.length; ) {
      let j = i;
      while (j + 1 < children.length && children[j + 1].name ===
children[i].name) j++;
      if (j - i >= 3) { walk(children[i], d); lines.push(pad(d) + `…
${children[i].name} ×${j - i} more`); }
      else for (let k = i; k <= j; k++) walk(children[k], d);
      i = j + 1;
    }
  };
  spans.filter((s) => !pid(s) || !ids.has(pid(s)!)).sort((a, b) => t(a) -
t(b)).forEach((r) => walk(r, 0));
  return lines.length > maxLines
```

```
    ? lines.slice(0, maxLines).join("\n") + `\n… ${lines.length - maxLines}
more lines truncated`
    : lines.join("\n");
}
```

`src/runner/runner.ts` starts one child per job, with the backend repo as working directory and the chosen environment's variables:

```
import { spawn } from "node:child_process";
import { mkdtempSync, readFileSync } from "node:fs";
import os from "node:os";
import path from "node:path";
import dotenv from "dotenv";
import { ROOT, cfg, type Env } from "./config.js";

export async function runJob(env: Env, job: Record<string, unknown>,
timeoutMs = 120_000) {
  const c = cfg(env);
  const backendEnv = dotenv.parse(readFileSync(path.join(ROOT,
c.backend_env_file)));
  const out = path.join(mkdtempSync(path.join(os.tmpdir(), "dbg-")),
"out.json");
  const child = spawn(path.join(ROOT, "node_modules/.bin/tsx"),
[path.join(ROOT, "src/runner/child.ts"), JSON.stringify({ ...job, out })], {
    cwd: process.env.ECOM_REPO,
    env: { ...process.env, ...backendEnv, DEBUG_RUNNER: "true",
ALLOW_POST_PATHS: JSON.stringify(c.allow_post_paths ?? []) },
    stdio: ["ignore", "ignore", "pipe"],
  });
  let stderr = "";
  child.stderr!.on("data", (d) => { stderr = (stderr + d).slice(-4000); });
  await new Promise((resolve, reject) => {
    const timer = setTimeout(() => { child.kill("SIGKILL"); reject(new
Error("runner timed out")); }, timeoutMs);
    child.on("exit", (code) => { clearTimeout(timer); code === 0 ?
resolve(null) : reject(new Error(`runner exit ${code}:
${stderr.slice(-1500)}`)); });
  });
  return JSON.parse(readFileSync(out, "utf8"));
}
```

And a small CLI for testing, `src/runner/cli.ts`:

```
import "dotenv/config";
import { runJob } from "./runner.js";
const [env, kind, a, b] = process.argv.slice(2);
const job = kind === "request" ? { kind, method: "GET", url: a }
          : kind === "worker" ? { kind, name: a, args: JSON.parse(b ?? "{}")
}
          : { kind };
const r = await runJob(env as any, job);
console.log(r.trace ?? ""); console.log("RESULT:", r.result); if (r.error)
console.log("ERROR:", r.error);
if (r.blocked_writes?.length) console.log("BLOCKED WRITES:\n" +
r.blocked_writes.join("\n"));
```

6.9 Verify

```
cd ~/work/ecom-backend && npm run build
cd ~/work/ecom-debug-bot
npm pkg set scripts.run-debug="tsx src/runner/cli.ts"
npm run run-debug -- stage routes                                    # every
Hapi route, from server.table()
npm run run-debug -- stage workers                                   # every
registered worker
npm run run-debug -- stage request "/products/4455/offers?storeId=12"
npm run run-debug -- stage worker offer-publisher '{"skus":["4455"]}'
```

You should see a trace tree like the one at the top of this phase, and for the worker a `BLOCKED WRITES` list showing exactly what it *would* have written. If the tree has few business lines, widen the `WATCH` folder list (environment variable `WATCH="offers|pricing|…"`) to match your `src/` layout.

When the trace isn't enough: if one big function makes the whole decision internally, you'll only see its input and output. Fix those gradually: extract the decision into a small exported function that returns a reason (for example `{ eligible: false, reason: "OFFER_NOT_STACKABLE", blockedBy: "OFF-099" }`). The runner picks it up automatically.

Phase 7 — The gateway: one MCP server, generic tools

MCP (Model Context Protocol) is a standard way to give an AI tool extra tools. The gateway is the single place the bot connects to. Its tools are generic, so they work for every route, worker and service without new code.

Tool	What it does	Typical question
`reproduce_request`	Runs a real Hapi request in an env through the runner, returns trace + response	"Why is offer X missing for SKU Y in stage?"
`run_worker`	Runs a cron worker for given SKUs (dry run), returns trace + blocked writes	"Why didn't the feed job publish SKU Y?"
`list_routes` / `list_workers`	Lists what exists (from `server.table()` and the registry)	"Which endpoint returns offers?"
`call_api`	Direct GET to a downstream service	"What does inventory say right now?"
`query_db`	Read-only SELECT on the env's replica	"What's stored for SKU Y?"
`read_knowledge`	Reads a `.knowledge/` file	Business rules, glossary, data meanings
`search_code` (Phase 9), `find_job_traces` (Phase 8.5)	Search code; past job traces	"Where do we handle BOGO?"

7.1 Create the server

`src/mcp/server.ts`:

```
import "dotenv/config";
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from
"@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";
import { readFileSync } from "node:fs";
```

```
import path from "node:path";
import axios from "axios";
import pg from "pg";
import { runJob } from "../runner/runner.js";
import { cfg, secrets, ALLOWED_ENVS } from "../runner/config.js";
import { mask } from "../runner/mask.js";

const KNOWLEDGE = path.join(process.env.ECOM_REPO!, ".knowledge");
const env = z.enum(ALLOWED_ENVS);
const out = (data: unknown) => ({ content: [{ type: "text" as const,
  text: (typeof data === "string" ? data :
JSON.stringify(mask(data))).slice(0, 12_000) }] });
const fail = (e: any) => out({ error: e?.response?.status ? `HTTP
${e.response.status}` : String(e?.message ?? e) });
const traced = (r: any) => out([
  r.error ? `ERROR: ${r.error}` : `RESULT: ${r.result}`,
  `TRACE:\n${r.trace}`,
  r.blocked_writes?.length ? `BLOCKED WRITES (would have
written):\n${r.blocked_writes.join("\n")}` : "",
].join("\n\n"));

const server = new McpServer({ name: "ecom-debug", version: "0.2.0" });

server.registerTool("reproduce_request", {
  description: "Run a real request through our Hapi server in the given env
(read-only) and return the response plus a trace of every business function,
downstream API call and DB query. Use first for any 'why does endpoint X
return Y' question.",
  inputSchema: { env, url: z.string().describe("path + query, e.g.
/products/4455/offers?storeId=12"),
    method: z.enum(["GET", "POST", "PUT", "DELETE"]).default("GET"),
    headers: z.record(z.string()).optional(), payload: z.any().optional() },
}, async ({ env, url, method, headers, payload }) => {
  try { return traced(await runJob(env, { kind: "request", url, method,
headers, payload })); } catch (e) { return fail(e); }
});

server.registerTool("run_worker", {
  description: "Run one cron worker in dry-run mode for specific SKUs in the
given env. Returns its trace and the writes it would have made.",
  inputSchema: { env, name: z.string(), skus: z.array(z.string()).optional(),
limit: z.number().int().max(100).optional() },
}, async ({ env, name, skus, limit }) => {
```

```
  try { return traced(await runJob(env, { kind: "worker", name, args: { skus,
limit } }, 300_000)); } catch (e) { return fail(e); }
});

server.registerTool("list_routes", {
  description: "List all Hapi routes (method, path, description).",
inputSchema: { env },
}, async ({ env }) => { try { return out((await runJob(env, { kind: "routes"
})).result); } catch (e) { return fail(e); } });

server.registerTool("list_workers", {
  description: "List all registered cron workers.", inputSchema: { env },
}, async ({ env }) => { try { return out((await runJob(env, { kind: "workers"
})).result); } catch (e) { return fail(e); } });

server.registerTool("call_api", {
  description: "GET a downstream service directly, e.g. service='inventory',
path='/inventory/4455', query={storeId:'12'}.",
  inputSchema: { env, service: z.string(), path: z.string(), query:
z.record(z.string()).optional() },
}, async ({ env, service, path: p, query }) => {
  const c = cfg(env);
  const base = c.services?.[service];
  if (!base) return out({ error: `Unknown service. Known:
${Object.keys(c.services ?? {}).join(", ")}` });
  const token = secrets(env)[c.auth?.token_env];
  try {
    const r = await axios.get(base + p, { params: query, timeout: 10_000,
headers: token ? { Authorization: `Bearer ${token}` } : {} });
    return out({ status: r.status, data: r.data });
  } catch (e) { return fail(e); }
});

server.registerTool("query_db", {
  description: "Run ONE read-only SQL SELECT on the env's read replica.
Returns at most 200 rows. Check .knowledge/data/data-dictionary.yaml for
table names first.",
  inputSchema: { env, sql: z.string() },
}, async ({ env, sql }) => {
  const clean = sql.trim().replace(/;\s*$/, "");
  if (!/^(select|with)\b/i.test(clean) || clean.includes(";") ||
/\b(insert|update|delete|drop|alter|truncate|grant|create|copy)\b/i.test(clea
n))
```

```
    return out({ error: "Only a single SELECT statement is allowed" });
  const client = new pg.Client({ connectionString: secrets(env)
[cfg(env).db_readonly_env] });
  try {
    await client.connect();
    await client.query("SET statement_timeout = 10000; SET
default_transaction_read_only = on;");
    const r = await client.query(`SELECT * FROM (${clean}) q LIMIT 200`);
    return out({ rowCount: r.rowCount, rows: r.rows });
  } catch (e) { return fail(e); } finally { await client.end().catch(() =>
{}); }
});

server.registerTool("read_knowledge", {
  description: "Read a file from the .knowledge folder, e.g. 'INDEX.md' or
'rules/offer-eligibility.yaml'.",
  inputSchema: { file: z.string() },
}, async ({ file }) => {
  const p = path.resolve(KNOWLEDGE, file);
  if (!p.startsWith(KNOWLEDGE)) return out({ error: "Path outside .knowledge"
});
  try { return out(readFileSync(p, "utf8")); } catch (e) { return fail(e); }
});

await server.connect(new StdioServerTransport());
console.error("ecom-debug gateway running");    // stderr only: stdout is the
MCP channel
```

In an MCP server over stdio, **stdout is the protocol channel**. Log with `console.error` only. The backend's own stdout logs are safe because they run in the runner's child process, whose stdout is discarded.

7.2 Save routes and workers as knowledge

The real route table is better than anything parsed from source. Add `src/runner/catalog.ts`:

```
import "dotenv/config";
import { writeFileSync, mkdirSync } from "node:fs";
import path from "node:path";
import YAML from "yaml";
import { runJob } from "./runner.js";

const dir = path.join(process.env.ECOM_REPO!, ".knowledge/generated");
mkdirSync(dir, { recursive: true });
writeFileSync(path.join(dir, "hapi-routes.yaml"), YAML.stringify((await
runJob("stage", { kind: "routes" })).result));
writeFileSync(path.join(dir, "workers.yaml"), YAML.stringify((await
runJob("stage", { kind: "workers" })).result));
console.log("route + worker catalog written");
```

```
npm pkg set scripts.catalog="tsx src/runner/catalog.ts"
npm run catalog
```

Add the cron schedule of each worker to `.knowledge/generated/workers.yaml` by hand once (or extract it from your cron file), so the bot knows when each one runs.

7.3 Test the tools without the AI tool

```
npm pkg set scripts.mcp="tsx src/mcp/server.ts"
npx @modelcontextprotocol/inspector npx tsx src/mcp/server.ts
```

The Inspector opens in your browser. Click **List Tools**, choose `reproduce_request`, enter `stage` and `/products/4455/offers?storeId=12`, and run it. Then try `run_worker` with a worker name and one SKU.

Verify: you get `RESULT`, a `TRACE` tree and, for workers, `BLOCKED WRITES`. Fix boot errors (missing env vars in `.env.backend.stage`, DB/Redis connection) here before connecting your AI tool. A stage reproduction usually takes 3–10 seconds, mostly server boot.

7.4 Connect the gateway to your approved AI tool

Register the gateway as a **local (stdio) MCP server** in the `ecom-backend` workspace, so the tool sees both your code and the bot's tools. Use absolute paths. Pick your tool:

GitHub Copilot (VS Code, agent mode) — create `ecom-backend/.vscode/mcp.json`:

```
{
  "servers": {
    "ecom-debug": {
      "type": "stdio",
      "command": "npx",
      "args": ["tsx", "/Users/you/work/ecom-debug-bot/src/mcp/server.ts"],
      "env": { "ECOM_REPO": "/Users/you/work/ecom-backend" }
    }
  }
}
```

Open Copilot Chat, switch to **Agent**, click the tools icon and enable the `ecom-debug` tools.

Cursor — create `ecom-backend/.cursor/mcp.json`:

```
{
  "mcpServers": {
    "ecom-debug": {
      "command": "npx",
      "args": ["tsx", "/Users/you/work/ecom-debug-bot/src/mcp/server.ts"],
      "env": { "ECOM_REPO": "/Users/you/work/ecom-backend" }
    }
  }
}
```

Then **Settings → MCP**: the server should show a green dot and its tool list. Use the chat in **Agent** mode.

Claude Code — create `ecom-backend/.mcp.json` with the same `mcpServers` block as Cursor, then run `claude` in `ecom-backend` and approve the server when asked. `/mcp` shows its status.

Windsurf / JetBrains AI Assistant / others — add the same `command`, `args` and `env` in the tool's MCP settings (Windsurf: `~/.codeium/windsurf/mcp_config.json`; JetBrains: Settings → Tools → AI Assistant → Model Context Protocol).

Notes:

- Commit the config file only if every teammate uses the same paths; otherwise add it to `.gitignore` and share a template.
- The tool will ask before each tool call. Every tool is read-only, so you can safely choose "always allow" for them.

- `run_worker` can take minutes. If your tool times out tool calls, pass `limit` (1–5 SKUs) to keep runs short.

7.5 Give the bot its instructions

Put the instructions in `AGENTS.md` at the root of `ecom-backend` (several tools read it automatically) and point your tool's own rules file at it. Replace the `AGENTS.md` from Phase 5.2 with:

```
# AGENTS.md
You debug our eCommerce backend (Hapi API + cron workers). Environments: dev,
stage, qa.

Knowledge: start with .knowledge/INDEX.md and load only the files it points
to.
Prefer YAML knowledge over source; open source only at the file#function a
rule cites.
Never edit .knowledge/generated/* by hand.

Debugging (ecom-debug tools):
1. If you don't know the endpoint or worker, call list_routes or
list_workers.
2. REPRODUCE the behaviour: reproduce_request for API questions, run_worker
for job questions.
   Ask for env, SKU, store or offer id if missing.
3. Read the TRACE top to bottom. Find the step where the expected item
disappears or a value changes.
4. Confirm suspicious data with call_api or query_db. Quote values from tool
output; never guess data.
5. If a tool errors, report the error and stop.
Answer format: Verdict (1 line) -> Where it happened (function + evidence) ->
All contributing causes -> Fix / next step.
Do not modify code while debugging unless asked.
```

Then add a one-line pointer for your tool, if it doesn't read `AGENTS.md` itself:

Tool	File	Content
GitHub Copilot	`.github/copilot-instructions.md`	`Follow the instructions in AGENTS.md.`

Tool	File	Content
Cursor	`.cursor/rules/ecom-debug.mdc` (set "Always apply")	`Follow the instructions in AGENTS.md.`
Claude Code	`CLAUDE.md`	`@AGENTS.md`

Verify — your target scenario: in agent mode ask "Offer OFF-123 is not showing for SKU 4455 in stage, store 12. Why?" The tool should read INDEX.md, find the offers route, call `reproduce_request`, and point to the exact functions where OFF-123 dropped out, quoting values from the trace. Follow up with "Is the offer-publisher job the cause?" and it should call `run_worker` for that SKU.

Phase 8 — Observe your data jobs

Three pieces: **traces** show what each job run did step by step, **reason-coded logs** explain why each record was classified or skipped, and a **profiler** records what the output data looks like.

8.1 Run Jaeger locally (trace viewer)

```
docker run -d --name jaeger --restart always \
  -e COLLECTOR_OTLP_ENABLED=true \
  -p 127.0.0.1:16686:16686 -p 127.0.0.1:4318:4318 \
  jaegertracing/all-in-one:latest
```

UI: `http://localhost:16686`. Port 4318 receives traces.

8.2 Add OpenTelemetry to the backend's jobs

```
cd ~/work/ecom-backend
npm i @opentelemetry/api @opentelemetry/sdk-node @opentelemetry/auto-
instrumentations-node \
      @opentelemetry/exporter-trace-otlp-http pino
```

Create `src/observability/tracing.ts`:

```
import { NodeSDK } from "@opentelemetry/sdk-node";
import { OTLPTraceExporter } from "@opentelemetry/exporter-trace-otlp-http";
import { getNodeAutoInstrumentations } from "@opentelemetry/auto-
instrumentations-node";

if (process.env.OTEL_ENABLED === "true") {
  const sdk = new NodeSDK({
    serviceName: process.env.OTEL_SERVICE_NAME ?? "ecom-jobs",
    traceExporter: new OTLPTraceExporter({ url:
"http://localhost:4318/v1/traces" }),
    instrumentations: [getNodeAutoInstrumentations()],   // auto-traces HTTP,
pg, mongodb, redis...
  });
  sdk.start();
  process.on("SIGTERM", () => sdk.shutdown());
}
```

Make it the **first import** in each job's entry file: `import "../observability/tracing";`

8.3 Add spans and reason codes inside a job

Create `src/observability/logger.ts`:

```
import pino from "pino";
export const logger = pino({ level: process.env.LOG_LEVEL ?? "info" });
```

Then wrap each stage and each classification decision. Example for a product-feed job:

```
import { trace } from "@opentelemetry/api";
import { randomUUID } from "node:crypto";
import { logger } from "../observability/logger";
const tracer = trace.getTracer("product-feed-job");

export async function runProductFeedJob() {
  const runId = randomUUID();
  const log = logger.child({ job: "product-feed", runId });
  await tracer.startActiveSpan("product-feed.run", async (run) => {
    run.setAttribute("job.runId", runId);
    const rows = await tracer.startActiveSpan("extract", async (s) => { const
r = await loadFeed(); s.setAttribute("records.in", r.length); s.end(); return
r; });

    for (const row of rows) {
```

```
      await tracer.startActiveSpan("classify", async (s) => {
        const result = classify(row);                       // your existing
logic
        s.setAttributes({ sku: row.sku, "classification.result":
result.category, "classification.reason": result.reason });
        log.info({ sku: row.sku, category: result.category, reason:
result.reason }, "classified");
        if (result.category === "SKIPPED") log.warn({ sku: row.sku, reason:
result.reason }, "skipped");
        s.end();
      });
    }
    run.end();
  });
}
```

Make `classify` return a `reason` string such as `SKIPPED_NO_PRICE` or `CLASSIFIED_AS_BUNDLE`, the same pattern as the offer rules. Write these codes into `.knowledge/data/job-outputs.yaml` (below) with a one-line meaning each.

Run the job with tracing and logs to a file:

```
OTEL_ENABLED=true npx tsx src/jobs/productFeed.ts | tee logs/product-
feed.jsonl
```

Verify: in Jaeger, choose service `ecom-jobs` → Find Traces. You should see one `product-feed.run` trace with `extract`, `classify` and auto-traced HTTP/DB child spans. Search by tag `sku=4455` to follow one product through the job.

Per-record spans can be large. For jobs over ~50k records, only create `classify` spans for skipped/unusual records and rely on logs for the rest.

8.4 Profile job outputs with DuckDB

In the tooling repo:

```
cd ~/work/ecom-debug-bot
npm i @duckdb/node-api
npm pkg set scripts.profile="tsx src/profile/index.ts"
```

Describe each job's outputs in `config/jobs.yaml`:

```
- job: product-feed
  outputs:
    - name: classified_products
      kind: file
      source: /data/feeds/out/classified/*.json       # json, csv or parquet
    - name: product_offers
      kind: postgres
      source: public.product_offers
      db_env: STAGE_DB_URL_READONLY
```

Create `src/profile/index.ts`:

```
import "dotenv/config";
import { DuckDBInstance } from "@duckdb/node-api";
import { readFileSync, writeFileSync, mkdirSync } from "node:fs";
import path from "node:path";
import dotenv from "dotenv";
import YAML from "yaml";

const jobs = YAML.parse(readFileSync("config/jobs.yaml", "utf8"));
const secrets = dotenv.parse(readFileSync(".env.stage"));
const db = await DuckDBInstance.create(":memory:");
const conn = await db.connect();
const q = async (sql: string) => (await
conn.runAndReadAll(sql)).getRowObjectsJson();

function fromClause(o: any) {
  if (o.kind === "file") {
    const ext = path.extname(o.source);
    const fn = ext === ".csv" ? "read_csv_auto" : ext === ".parquet" ?
"read_parquet" : "read_json_auto";
    return `${fn}('${o.source}')`;
  }
  return `pg.${o.source}`;
}

const report: any[] = [];
for (const job of jobs) {
  for (const o of job.outputs) {
    if (o.kind === "postgres") {
      await q("INSTALL postgres; LOAD postgres;");
      await q("DETACH DATABASE IF EXISTS pg;");
```

```
      await q(`ATTACH '${secrets[o.db_env]}' AS pg (TYPE postgres,
READ_ONLY);`);
    }
    const src = `(SELECT * FROM ${fromClause(o)} USING SAMPLE 20000 ROWS)`;
    const summary = await q(`SUMMARIZE SELECT * FROM ${src}`);
    const columns: any[] = [];
    for (const c of summary as any[]) {
      const col: any = { name: c.column_name, type: c.column_type, null_pct:
c.null_percentage, distinct_approx: c.approx_unique, min: c.min, max: c.max
};
      if (Number(c.approx_unique) <= 20) {     // looks like a category/enum
-> show its values
        col.top_values = await q(`SELECT "${c.column_name}" AS v, count(*) AS
n FROM ${src} GROUP BY 1 ORDER BY 2 DESC LIMIT 10`);
      }
      columns.push(col);
    }
    report.push({ job: job.job, output: o.name, source: o.source,
profiled_at: new Date().toISOString(), columns });
    console.log("profiled", job.job, o.name);
  }
}
const outDir = path.join(process.env.ECOM_REPO!, ".knowledge/data");
mkdirSync(outDir, { recursive: true });
writeFileSync(path.join(outDir, "job-outputs.generated.yaml"),
YAML.stringify(report));
```

```
npm run profile
```

`INSTALL postgres` downloads a DuckDB extension once. On a fully offline machine, run it once while connected, or copy the extension file from another machine.

Verify: `.knowledge/data/job-outputs.generated.yaml` lists every column with type, null %, distinct count, and the value distribution of category-like columns (for example how many products landed in each classification). Keep a hand-written `job-outputs.yaml` next to it with the meaning of each reason code and category.

8.5 Let the bot read traces

Add one more tool to `src/mcp/server.ts`:

```
server.registerTool("find_job_traces", {
  description: "Find recent job traces in Jaeger for a SKU, to see how a data
job processed/classified it.",
  inputSchema: { sku: z.string(), service: z.string().default("ecom-jobs") },
}, async ({ sku, service }) => {
  const tags = encodeURIComponent(JSON.stringify({ sku }));
  const res = await fetch(`http://localhost:16686/api/traces?
service=${service}&tags=${tags}&limit=5`);
  const { data } = await res.json();
  const spans = data.flatMap((t: any) => t.spans.map((s: any) => ({
    trace: t.traceID, op: s.operationName, start: new Date(s.startTime /
1000).toISOString(),
    tags: Object.fromEntries(s.tags.map((x: any) => [x.key, x.value])) })));
  return out(spans.slice(0, 50));
});
```

8.6 Optional: Metabase for visual exploration

```
docker run -d --name metabase -p 127.0.0.1:3001:3000 -e
MB_ANON_TRACKING_ENABLED=false metabase/metabase
```

Open `http://localhost:3001`, connect the read-only stage database, and build quick charts of classification counts per run. This is for you to understand the logic before documenting it.

Phase 9 — Code and knowledge search (no model needed)

The bot needs to find code for questions like "where do we handle BOGO?". Instead of embeddings (which need a model), this phase builds a keyword index with **MiniSearch**: a small JavaScript library that ranks results by relevance, tolerates typos, splits `camelCase` names, and expands your own synonyms. The index is one JSON file; there's no server to run.

9.1 Install

```
cd ~/work/ecom-debug-bot
npm i minisearch
npm pkg set scripts.index="tsx src/search/index.ts"
printf ".search-index.json\n" >> .gitignore
```

9.2 Shared search settings

Create `src/search/options.ts` (the indexer and the search tool must use identical options):

```
// Split identifiers so "resolveStack" also matches "resolve" and "stack"
const tokenize = (text: string) =>
  text.split(/[^A-Za-z0-9]+/).filter(Boolean)
    .flatMap((t) => [t, ...t.split(/(?<=[a-z0-9])(?=[A-Z])/)]);

export const SEARCH_OPTIONS = {
  idField: "id",
  fields: ["symbol", "file", "text"],
  storeFields: ["file", "symbol", "text"],
  tokenize,
  processTerm: (t: string) => (t.length < 2 ? null : t.toLowerCase()),
  searchOptions: { boost: { symbol: 3, file: 1.5 }, fuzzy: 0.2, prefix: true,
combineWith: "OR" },
};
```

9.3 Add synonyms for business words

Keyword search only matches words that appear in the code, so teach it your vocabulary. Create `ecom-backend/.knowledge/search-synonyms.yaml`:

```
bogo: [buy one get one, buyOneGetOne, bxgy]
stock: [inventory, availableQty, availability]
discount: [offer, promotion, promo, markdown]
stackable: [stacking, combinable, resolveStack]
ecom: [ecomFlag, online, sellable]
```

Add a line whenever a search misses something it should find.

9.4 Build the index

Create `src/search/index.ts` (one entry per function, one per knowledge section):

```
import "dotenv/config";
import MiniSearch from "minisearch";
import { Project } from "ts-morph";
import fg from "fast-glob";
import { readFileSync, writeFileSync } from "node:fs";
import path from "node:path";
```

```
import { SEARCH_OPTIONS } from "./options.js";

const REPO = process.env.ECOM_REPO!;
const docs: { id: string; file: string; symbol: string; text: string }[] =
[];
const seen = new Map<string, number>();
const add = (file: string, symbol: string, text: string) => {
  const base = `${file}#${symbol}`;
  const n = (seen.get(base) ?? 0) + 1;
  seen.set(base, n);
  docs.push({ id: n === 1 ? base : `${base}~${n}`, file, symbol, text:
text.slice(0, 4000) });
};

// Code: one entry per function / method
const project = new Project({ tsConfigFilePath: path.join(REPO,
"tsconfig.json") });
for (const sf of project.getSourceFiles()) {
  const file = path.relative(REPO, sf.getFilePath());
  if (/node_modules|\.d\.ts$|\.(spec|test)\.ts$/.test(file)) continue;
  for (const fn of [...sf.getFunctions(), ...sf.getClasses().flatMap((c) =>
c.getMethods())]) {
    add(file, fn.getName() ?? "anonymous", fn.getText());
  }
}

// Knowledge: one entry per markdown section / yaml file
for (const f of await fg(["**/*.md", "**/*.yaml"], { cwd: path.join(REPO,
".knowledge"), ignore: ["generated/**", "prompts/**"] })) {
  const text = readFileSync(path.join(REPO, ".knowledge", f), "utf8");
  text.split(/\n(?=## )/).forEach((part) => add(`.knowledge/${f}`,
part.split("\n")[0].slice(0, 80), part));
}

const ms = new MiniSearch(SEARCH_OPTIONS);
ms.addAll(docs);
writeFileSync(path.resolve(".search-index.json"), JSON.stringify(ms));
console.error(`indexed ${docs.length} entries`);
```

```
npm run index
```

This takes seconds, even for a large repo.

9.5 Add the search tool to the gateway

In `src/mcp/server.ts` add:

```
import MiniSearch from "minisearch";
import YAML from "yaml";
import { SEARCH_OPTIONS } from "../search/options.js";

const INDEX_FILE = path.resolve(import.meta.dirname, "../../.search-
index.json");
const synonyms = (): Record<string, string[]> => {
  try { return YAML.parse(readFileSync(path.join(KNOWLEDGE, "search-
synonyms.yaml"), "utf8")) ?? {}; } catch { return {}; }
};

server.registerTool("search_code", {
  description: "Keyword search over source functions and knowledge notes
(typo-tolerant, synonym-aware). Returns file#function and a snippet.",
  inputSchema: { query: z.string(), limit:
z.number().int().max(10).default(5) },
}, async ({ query, limit }) => {
  const ms = MiniSearch.loadJSON(readFileSync(INDEX_FILE, "utf8"),
SEARCH_OPTIONS);
  const syn = synonyms();
  const words = query.toLowerCase().split(/\s+/);
  const expanded = [query, ...words.flatMap((w) => syn[w] ?? [])].join(" ");
  const hits = ms.search(expanded).slice(0, limit);
  return out(hits.map((h: any) => ({ id: h.id, score: Math.round(h.score),
snippet: String(h.text).slice(0, 600) })));
});
```

Verify: in the Inspector (Phase 7.3) run `search_code` with "bogo". The top hits should be your BOGO/promotion functions. Snippets are capped at 600 characters to keep token use low; the AI tool can open the full file itself.

9.6 Optional: a shared index in Redis or CockroachDB

The JSON index works per laptop. If your team wants one shared, always-current index, store the same entries (`id` , `file` , `symbol` , `text`) in a database with full-text search and point `search_code` at it:

- **Redis** with its query engine: create an index over hashes with `FT.CREATE ... SCHEMA symbol TEXT WEIGHT 3 file TEXT text TEXT` , then query with `FT.SEARCH` .

- **CockroachDB**: a table with a generated `tsvector` column plus an inverted index, queried with `@@ plainto_tsquery(...)`.

Both give keyword search, not meaning-based search, so keep the synonyms file either way. Neither replaces the AI tool; they only store and find text.

Phase 10 — Keep the knowledge fresh automatically

Scripts keep the generated facts, search index and data profiles current. Module summaries are the only part that needs the AI tool, so a script lists which ones are stale and you refresh just those.

What	When	Who
`extract` (facts) + `check-rules`	every commit (pre-commit hook)	script, seconds
`stale` + `index` + `profile` + `catalog`	nightly, 02:00	script, minutes
Refresh module summaries listed in `STALE.md`	when the list isn't empty	you + AI tool (prompt from 4.4)
Review hand-written rules	when `check-rules` fails or offer code changes	you

10.1 List modules whose summaries are stale

Create `src/extract/stale.ts` in the tooling repo. It compares each summary's `source_hash` comment (written by the 4.4 prompt) with the current hashes from `manifest.json`:

```
import "dotenv/config";
import { readFileSync, writeFileSync, existsSync } from "node:fs";
import { createHash } from "node:crypto";
import path from "node:path";

const K = path.join(process.env.ECOM_REPO!, ".knowledge");
const manifest: Record<string, string> = JSON.parse(readFileSync(path.join(K,
"generated/manifest.json"), "utf8"));
```

```
// one hash per module = hash of its files' hashes
const modules: Record<string, string[]> = {};
for (const [file, h] of Object.entries(manifest)) (modules[file.split("/")[1]
?? "root"] ??= []).push(file + h);

const stale: string[] = [];
for (const [mod, parts] of Object.entries(modules)) {
  const current =
createHash("sha1").update(parts.sort().join()).digest("hex").slice(0, 12);
  const md = path.join(K, "modules", `${mod}.md`);
  const recorded = existsSync(md) ? readFileSync(md,
"utf8").match(/source_hash:\s*(\w+)/)?.[1] : undefined;
  if (recorded !== current) stale.push(`- ${mod} (source_hash should be
${current})`);
}
writeFileSync(path.join(K, "STALE.md"),
  stale.length ? `# Module summaries to refresh\nRun
.knowledge/prompts/module-summary.md for:\n${stale.join("\n")}\n` : "# All
module summaries are current\n");
console.log(`${stale.length} stale module summaries`);
```

The 4.4 prompt already tells the AI tool to copy that value into each summary: `At the very top, add: <!-- source_hash: {the value listed for this module in .knowledge/STALE.md} -->`. To refresh, tell your AI tool: "Refresh every module listed in .knowledge/STALE.md using .knowledge/prompts/module-summary.md", then review the diffs.

10.2 Check that hand-written rules still point at real code

Create `src/extract/check-rules.ts`:

```
import "dotenv/config";
import { readFileSync, existsSync } from "node:fs";
import path from "node:path";
import YAML from "yaml";

const REPO = process.env.ECOM_REPO!;
const rules = YAML.parse(readFileSync(path.join(REPO,
".knowledge/rules/offer-eligibility.yaml"), "utf8")).rules;
let bad = 0;
for (const r of rules) {
  const [file, fn] = r.source.split("#");
  const p = path.join(REPO, file);
  if (!existsSync(p) || (fn && !readFileSync(p, "utf8").includes(fn))) {
    console.error(`STALE RULE ${r.id}: ${r.source} not found`); bad++;
  }
}
process.exit(bad ? 1 : 0);
```

Add the scripts:

```
cd ~/work/ecom-debug-bot
npm pkg set scripts.check-rules="tsx src/extract/check-rules.ts"
npm pkg set scripts.stale="tsx src/extract/stale.ts"
npm pkg set scripts.nightly="npm run extract && npm run stale && npm run
index && npm run profile && npm run catalog"
```

10.3 Pre-commit hook in the backend (Husky)

```
cd ~/work/ecom-backend
npm i -D husky
npx husky init
cat > .husky/pre-commit <<'EOF'
npm --prefix ../ecom-debug-bot run extract
npm --prefix ../ecom-debug-bot run check-rules
git add .knowledge/generated
EOF
```

This assumes both repos sit side by side in `~/work`. Teammates without the tooling repo can skip the hook with `git commit --no-verify`, or you can move these two scripts into the backend repo later.

10.4 Nightly job on your machine

```
crontab -e
# add this line (runs at 02:00 every day):
0 2 * * * cd ~/work/ecom-debug-bot && /usr/bin/env bash -lc 'npm run nightly'
>> ~/work/ecom-debug-bot/nightly.log 2>&1
```

On macOS, give your terminal app Full Disk Access if cron can't read the repos.

10.5 CI guard (optional)

In your backend CI, add a step that fails a pull request when offer code changes but the rules file doesn't:

```
if git diff --name-only origin/main...HEAD | grep -q '^src/offers/' && \
   ! git diff --name-only origin/main...HEAD | grep -q '^.knowledge/rules/';
then
  echo "Offer logic changed: please review .knowledge/rules/offer-
eligibility.yaml"; exit 1
fi
```

Verify: rename a function referenced in the rules file and try to commit; the hook should print `STALE RULE` and block the commit. Then change any file under `src/offers/`, run `npm run stale`, and `offers` should appear in `.knowledge/STALE.md`.

Troubleshooting and progress checklist

Symptom	Likely cause	Fix
AI tool answers without calling tools	Not in agent mode, or tools not enabled	Switch to Agent mode; enable the ecom-debug tools; check the tool reads `AGENTS.md` (Phase 7.5)
AI tool invents data	Tool error ignored	Check the call in the Inspector (7.3); keep the "never guess data" rule in `AGENTS.md`
Server not listed / red status in the tool	Wrong path or `npx` not on the tool's PATH	Use absolute paths; use the full path to `npx` (`which npx`); restart the editor

Symptom	Likely cause	Fix
MCP option missing in the tool	Disabled by your org	Ask your admin to allow MCP servers (Phase 1)
MCP server disconnects immediately	Something printed to stdout	Use `console.error` in the tooling repo (backend logs are safe inside the runner's child process)
`run_worker` times out in the tool	Tool's call timeout shorter than the worker	Pass `skus` and `limit: 1`; split the question into smaller runs
`Cannot find module '@/...'` when the runner loads dist/	Backend path aliases	Reuse whatever production uses to resolve aliases (tsc-alias or module-alias)
HTTP 401/403 from stage APIs	Wrong or expired token	Refresh the token in `.env.stage`; confirm it's a read-only service token
Trace stops early; call listed under BLOCKED WRITES	A downstream POST that only reads (search APIs) was blocked	Add its path to `allow_post_paths` in environments.yaml (Phase 6.3)
`search_code` misses an obvious function	Business word not in the code	Add it to `search-synonyms.yaml` (9.3) and run `npm run index`
Summaries are wrong	AI tool misread code	Fix the `.md` by hand; it's only rewritten when listed in `STALE.md`
Empty Jaeger	`OTEL_ENABLED` not set, or tracing not imported first	Check both; confirm port 4318 is mapped

Checklist

- [ ] Phase 1: approved AI tool has agent mode + MCP; it can read `AGENTS.md`
- [ ] Phase 2: security has OK'd masked stage/qa data going to the tool; prod excluded
- [ ] Phase 3: tooling repo created, dependencies installed
- [ ] Phase 4: `generated/` files produced; module summaries written and reviewed

- [ ] Phase 5: INDEX, rules, environments, glossary written
- [ ] Phase 6: createServer + worker registry merged, runner CLI prints a trace
- [ ] Phase 7: `reproduce_request` and `run_worker` work in the Inspector, then in your AI tool
- [ ] Phase 8: one job traced in Jaeger, outputs profiled
- [ ] Phase 9: `search_code` finds the right functions
- [ ] Phase 10: pre-commit hook and nightly job running; `STALE.md` empty

Once the offer flow works end to end, extending to the next domain (pricing, promotions, cart, another worker) needs no new tools: add its folder to `WATCH`, register any new worker in the registry, and write its rules YAML, synonyms and module notes so the AI tool can explain what the trace shows.